

Resolução de Equações Transcendentes

Métodos Numéricos

Bacharelado em Ciência da Computação

UNIFAL-MG

$$e^x - 3x$$

“God made the integers, all else is the work of man.” — Leopold Kronecker

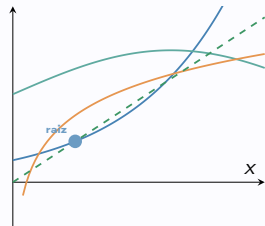
- 1 Introdução e Motivação
- 2 Adaptação dos Métodos Clássicos
- 3 Método do Ponto Fixo
- 4 Raízes Múltiplas em Funções Transcendentes
- 5 Otimização Unidimensional
- 6 Sistemas de Equações Transcendentes
- 7 Projeto: Equação de Kepler
- 8 Exercícios e Resumo

Definição

Uma função f é **transcendente** se não pode ser expressa como solução de equação polinomial com coeficientes racionais.

Exemplos:

- ▶ **Exponenciais:** e^x , a^x .
- ▶ **Logarítmicas:** $\ln(x)$, $\log_a(x)$.
- ▶ **Trigonométricas:** $\sin(x)$, $\cos(x)$, $\tan(x)$.
- ▶ **Hiperbólicas:** $\sinh(x)$, $\cosh(x)$.
- ▶ **Especiais:** Bessel, Gamma, Erf, ...



$$e^x = 3x \Rightarrow x \approx 0.619.$$

Boa Notícia!

Os métodos do Capítulo 4 (bisseção, Newton, secante) se aplicam **sem modificações conceituais** a equações transcendentais.

Aplicação 1: Criptografia de Curvas Elípticas (ECC)

Curvas elípticas são a base de **Bitcoin, TLS/SSL, assinaturas digitais**.

Operações em corpos finitos requerem resolver:

$$y^2 = x^3 + ax + b \pmod{p}.$$

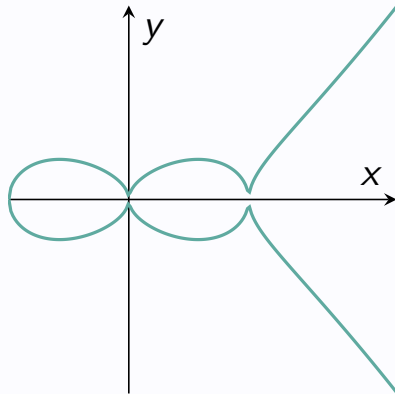
Curva secp256k1 (Bitcoin):

$$y^2 = x^3 + 7 \pmod{2^{256} - 2^{32} - 977}.$$

Desafios Numéricos

- ▶ Raízes de polinômios modulares.
- ▶ Logaritmos discretos (computacionalmente difícil).
- ▶ Funções elípticas (transcendentes).

$$y^2 = x^3 - x \text{ (exemplo)}$$



Refs: Koblitz (1987); Bitcoin Wiki (2024); Hankerson et al. (2004).

Concentração de portadores em equilíbrio:

$$n = N_c F_{1/2}(\eta), \quad F_{1/2}(\eta) = \frac{2}{\sqrt{\pi}} \int_0^\infty \frac{\sqrt{E}}{1 + e^{E-\eta}} dE.$$

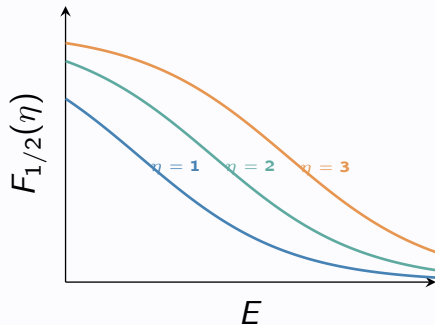
Problema: dado n , encontrar η (nível de Fermi) requer resolver:

$$n - N_c F_{1/2}(\eta) = 0.$$

Sem solução analítica! Newton-Raphson com aproximações para $F_{1/2}$.

Aplicações (Simuladores TCAD)

- ▶ Transistores: MOSFETs, FinFETs.
- ▶ Células solares.
- ▶ LEDs e lasers semicondutores.
- ▶ Sensores eletrônicos.



Ref: Sze and Ng (2006).

Equação de van der Waals para gases reais:

$$\left(P + \frac{a}{V^2}\right)(V - b) = RT.$$

Encontrar V dado P, T :

$$V^3 - \left(b + \frac{RT}{P}\right)V^2 + \frac{a}{P}V - \frac{ab}{P} = 0.$$

Para equações mais complexas (Peng-Robinson, SRK):

$$f(V, P, T) = 0 \quad (\text{transcendente}).$$

Resolvida milhares de vezes/segundo em simuladores de processos.

CO₂ — Dados

a	3.658 atm·L ² /mol ²
b	0.04267 L/mol
R	0.08206 atm·L/(mol·K)

Exercício

Para $T = 300$ K, $P = 10$ atm, encontre V usando Newton.

Ref: Smith et al. (2005).

Theorem (Aplicabilidade dos Métodos)

Os métodos clássicos se aplicam a funções transcendentais sob:

1. **Bisseção:** f contínua em $[a, b]$, $f(a)f(b) < 0$.
2. **Newton-Raphson:** $f \in C^1$, $f'(x)$ computável.
3. **Secante:** f contínua.
4. **Ponto Fixo:** $f(x) = 0$ reformulável como $x = g(x)$, g contínua.

✓ Boa Notícia

Funções transcendentais usuais (e^x , $\ln x$, $\sin x$, etc.) são **infinitamente diferenciáveis** em seus domínios, satisfazendo todos os requisitos acima.

Exemplo 1: $e^x = 3x$

$$f(x) = e^x - 3x, f'(x) = e^x - 3.$$

Raízes em $(0.5, 1)$ e $(1, 2)$.

Exemplo 2: $\cos(x) = x$

$$f(x) = \cos(x) - x, f'(x) = -\sin(x) - 1.$$

Raiz única em $[0, 1]$.

$e^x = 3x$ — Newton

$$x_{k+1} = x_k - \frac{e^{x_k} - 3x_k}{e^{x_k} - 3}$$

Com $x_0 = 0.7$:

$$x_1 \approx 0.6127$$

$$x_2 \approx 0.6191$$

$$x_3 \approx 0.6191 \quad \checkmark$$

Verificação: $e^{0.619} \approx 1.857 \approx 3 \times 0.619$

$\cos(x) = x$ — Constante de Dottie

$$x_{k+1} = x_k + \frac{\cos(x_k) - x_k}{\sin(x_k) + 1}$$

Com $x_0 = 0.5$:

$$x_1 \approx 0.7552$$

$$x_2 \approx 0.7391$$

$$x_3 \approx 0.7391 \quad \checkmark$$

$x^* \approx 0.7390851332 \dots$ (Constante de Dottie)

Implementação com SciPy

```
1 import numpy as np
2 from scipy.optimize import fsolve, brentq
3
4 # Exemplo 1:  $e^x = 3x$ 
5 def f1(x): return np.exp(x) - 3*x
6 def df1(x): return np.exp(x) - 3
7
8 def newton(f, df, x0, tol=1e-10, max_iter=100):
9     x = x0
10    for k in range(max_iter):
11        fx = f(x)
12        if abs(fx) < tol: return x, k+1
13        x = x - fx / df(x)
14    return x, max_iter
15
16 raiz1, iters1 = newton(f1, df1, 0.7)
17 print(f" $e^x = 3x$ : raiz = {raiz1:.10f}, iteracoes = {iters1}")
18
19 # Exemplo 2:  $\cos(x) = x$ 
20 def f2(x): return np.cos(x) - x
21 def df2(x): return -np.sin(x) - 1
22
23 raiz2, iters2 = newton(f2, df2, 0.5)
24 print(f" $\cos(x) = x$ : raiz = {raiz2:.10f}, iteracoes = {iters2}")
25
26 # Verificacao com SciPy
27 print(f"SciPy brentq: {brentq(f2, 0, 1):.10f}")
```

Formulação do Ponto Fixo

Resolver $f(x) = 0 \iff$ encontrar $x = g(x)$:

- ▶ $f(x) = x^2 - 2 = 0 \Rightarrow x = 2/x$.
- ▶ $f(x) = e^x - 3x = 0 \Rightarrow x = e^x/3$ ou $x = \ln(3x)$.

Iteração: $x^{(k+1)} = g(x^{(k)})$.

Definition (Ponto Fixo)

x^* é ponto fixo de g se $g(x^*) = x^*$.

Theorem (Brouwer — Existência)

Se $g : [a, b] \rightarrow [a, b]$ é contínua, então g possui pelo menos um ponto fixo em $[a, b]$.

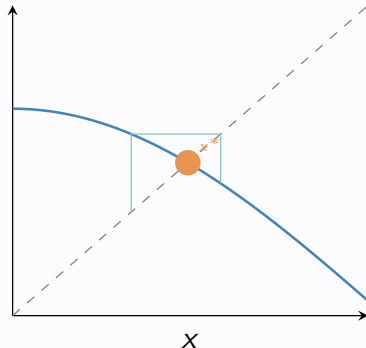


Diagrama de teia de aranha para $g(x) = \cos(x)$

Theorem (Convergência — Mapa de Contração)

Se $g : [a, b] \rightarrow [a, b]$ é diferenciável com $|g'(x)| \leq L < 1$ para todo $x \in [a, b]$, então:

1. Ponto fixo x^* é **único** em $[a, b]$
2. A iteração converge para qualquer $x^{(0)} \in [a, b]$:

$$|e^{(k+1)}| \leq L |e^{(k)}|$$

3. Se $g'(x^*) = 0$: convergência **quadrática**!

Exemplo — $\sqrt{2}$ por Ponto Fixo

$g_1(x) = 2/x$: $|g'_1(\sqrt{2})| = 1$ — **não converge!**

Método babilônico:

$$g_2(x) = \frac{1}{2} \left(x + \frac{2}{x} \right)$$

$g'_2(\sqrt{2}) = 0$ — **convergência quadrática!**

$$1 \rightarrow 1.5 \rightarrow 1.41667 \rightarrow 1.41422 \approx \sqrt{2}$$

* Interpretação Geométrica

$|g'(x)| < 1$: a função “achata” distâncias.

$|g'(x^*)| > 1$ diverge

$|g'(x^*)| = 1$ análise caso a caso

$|g'(x^*)| < 1$ converge

$g'(x^*) = 0$ conv. quadrática

Theorem (Newton $\equiv$ Ponto Fixo Quadrático)

Newton para $f(x) = 0$ equivale a ponto fixo com:

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

Consequência:

$$g'(x) = \frac{f(x) f''(x)}{[f'(x)]^2} \Rightarrow g'(x^*) = 0.$$

Logo, Newton é automaticamente de convergência quadrática!

Prova (cálculo de g')

$$g'(x) = 1 - \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}.$$

Em x^* : $f(x^*) = 0 \Rightarrow g'(x^*) = 0$. ✓

Escolha de g

Diferentes reformulações $f(x) = 0 \Leftrightarrow x = g(x)$ têm taxas diferentes! Sempre verificar $|g'(x^*)|$ antes de usar.

Implementação do Ponto Fixo

```
1 def ponto_fixo(g, x0, tol=1e-10, max_iter=100):
2     """Encontra ponto fixo de g via iteracao  $x_{k+1} = g(x_k)$ ."""
3     x = x0
4     historico = [x]
5     for k in range(max_iter):
6         x_new = g(x)
7         historico.append(x_new)
8         if abs(x_new - x) < tol:
9             return x_new, k+1, historico
10        x = x_new
11    print("Aviso: convergencia nao atingida")
12    return x, max_iter, historico
13
14 # sqrt(2) usando  $g(x) = (x + 2/x)/2$ 
15 g_sqrt2 = lambda x: 0.5 * (x + 2/x)
16 raiz, iters, hist = ponto_fixo(g_sqrt2, 1.0)
17 print(f"sqrt(2): {raiz:.15f}, iteracoes: {iters}")
18 print(f"Erro: {abs(raiz - np.sqrt(2)):.2e}")
19
20 # Mostrar convergencia quadratica
21 for i, xi in enumerate(hist[:6]):
22     print(f"  x[{i}] = {xi:.10f}, erro = {abs(xi - np.sqrt(2)):.2e}")
```

Theorem (Newton para Raízes Múltiplas — Revisão)

Se x^* é raiz de multiplicidade m de f (transcendente):

$$f(x) = (x - x^*)^m h(x), \quad h(x^*) \neq 0.$$

Newton padrão: convergência **linear** com fator $(m - 1)/m$.

Método modificado:

$$x_{k+1} = x_k - m \frac{f(x_k)}{f'(x_k)}.$$

restaura convergência **quadrática**.

Exemplo: $(x - 2)^2$ (revisão)

$m = 2$, Newton padrão $\Rightarrow$ convergência lenta.

Newton modificado ($m = 2$): converge em 1 passo.

! Problema Prático

Nem sempre conhecemos m a priori.

Estratégias:

- ▶ Usar $u(x) = f(x)/f'(x)$ (raízes de u são simples!).
- ▶ Detectar convergência lenta e estimar m .

Encontrar extremos de $f(x) \iff$ resolver $f'(x) = 0$.

Aplicar Newton a $g(x) = f'(x)$:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

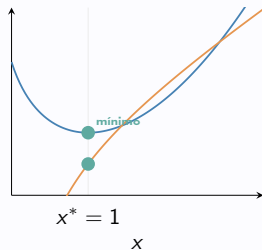
Minimizar $f(x) = x^2 - 2\ln(x)$, $x > 0$

$$f'(x) = 2x - 2/x = 0 \Rightarrow x = 1$$

Newton aplicado a f' :

$$x_{k+1} = x_k - \frac{x_k^3 - 1}{x_k^2 + 1},$$

com $x_0 = 2$: $2 \rightarrow 0.6 \rightarrow 1.176 \rightarrow 1.008 \rightarrow 1.000 \checkmark$



Verificar Tipo de Extremo

$$f''(x) = 2 + 2/x^2 > 0 \quad \forall x > 0$$

Logo $x = 1$ é **mínimo global**.

Resolver $F(x) = 0$, $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$:

$$\begin{cases} f_1(x_1, \dots, x_n) = 0 \\ \vdots \\ f_n(x_1, \dots, x_n) = 0 \end{cases}$$

Theorem (Newton para Sistemas)

$$x^{(k+1)} = x^{(k)} - J_F^{-1}(x^{(k)}) F(x^{(k)})$$

Jacobiana:

$$J_F = \begin{bmatrix} \partial f_1 / \partial x_1 & \cdots & \partial f_1 / \partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_n / \partial x_1 & \cdots & \partial f_n / \partial x_n \end{bmatrix}$$

Implementação eficiente: resolver sistema linear

$$J \Delta x = -F$$

Sistema 2×2

$$\begin{cases} x^2 + y^2 = 4 \\ e^x + y = 1 \end{cases}$$

$$J = \begin{bmatrix} 2x & 2y \\ e^x & 1 \end{bmatrix}$$

Custo

Cada iteração requer resolver um sistema linear $n \times n$ (custo $O(n^3)$).

```
1 from scipy.optimize import fsolve
2 import numpy as np
3
4 # Sistema:  $x^2 + y^2 = 4$ ,  $e^x + y = 1$ 
5 def sistema(vars):
6     x, y = vars
7     return [x**2 + y**2 - 4,
8             np.exp(x) + y - 1]
9
10 # Jacobiana analitica
11 def jacobiana(vars):
12     x, y = vars
13     return [[2*x, 2*y],
14             [np.exp(x), 1]]
15
16 sol1 = fsolve(sistema, [1, 1], fprime=jacobiana)
17 sol2 = fsolve(sistema, [-2, 0], fprime=jacobiana)
18
19 print(f"Solucao 1: x = {sol1[0]:.6f}, y = {sol1[1]:.6f}")
20 print(f"Solucao 2: x = {sol2[0]:.6f}, y = {sol2[1]:.6f}")
21
22 # Verificacao
23 for s in [sol1, sol2]:
24     r = sistema(s)
25     print(f"Residuo: {np.linalg.norm(r):.2e}")
```

Equação de Kepler:

$$E - e \sin(E) = M$$

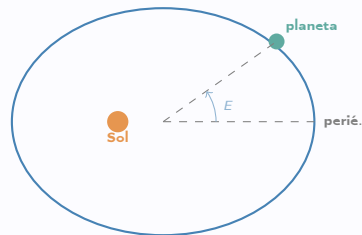
- ▶ E : anomalia excêntrica (incógnita)
- ▶ e : excentricidade ($0 < e < 1$)
- ▶ M : anomalia média (dado)

Newton para Kepler

$$f(E) = E - e \sin(E) - M, \quad f'(E) = 1 - e \cos(E):$$

$$E_{k+1} = E_k - \frac{E_k - e \sin(E_k) - M}{1 - e \cos(E_k)}$$

Converge em ~ 3 iterações para $e < 0.9$!



Terra ($e = 0.0167$)

Para $M = \pi/4$: resolve Newton em 3–4 iterações com $\varepsilon = 10^{-10}$.

Ref: Danby (1988).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def kepler_newton(M, e, tol=1e-12, max_iter=50):
5     """Resolve  $E - e \sin(E) = M$  para  $E$ ."""
6     E = M # chute inicial: anomalia media
7     for k in range(max_iter):
8         f = E - e * np.sin(E) - M
9         df = 1 - e * np.cos(E)
10        dE = f / df
11        E -= dE
12        if abs(dE) < tol:
13            return E, k+1
14    return E, max_iter
15
16 # Orbita da Terra: e = 0.0167
17 e = 0.0167
18 M_vals = np.linspace(0, 2*np.pi, 500)
19 E_vals = np.array([kepler_newton(M, e)[0] for M in M_vals])
20
21 # Verificacao: maximo residuo
22 residuo = np.max(np.abs(E_vals - e*np.sin(E_vals) - M_vals))
23 print(f"Residuo maximo: {residuo:.2e}") # deve ser < 1e-11
```

1. Resolva as equações com Newton-Raphson ($\varepsilon = 10^{-10}$) e compare com bisseção:
▷ $e^x = 3x$; $\sin(x) = x/2$; $x \ln(x) = 1$; $\tan(x) = x$
2. Demonstre que $f(x) = e^x - ax$ com $a > e$ tem exatamente duas raízes. Encontre-as para $a = 10$.
3. Teste ponto fixo para $x = \cos(x)$ com: $g_1(x) = \cos(x)$, $g_2(x) = \arccos(x)$, $g_3(x) = x - (\cos x - x)/(\sin x + 1)$. Qual converge mais rápido?
4. Para $f(x) = (x - 1)^3$ (raiz tripla): compare Newton padrão, Newton com $m = 3$, e método de Halley.
5. **Projeto Kepler:** para $e = 0.0167$, implemente solução para todo $M \in [0, 2\pi]$, plote $E \times M$ e compare com série de Fourier.
6. Resolva o sistema $\{x^2 - y^2 = 1, xy = 2\}$ via Newton multivariável. Encontre todas as soluções reais.

Conceitos Principais

- ▶ Funções transcendentais: definição e exemplos.
- ▶ Métodos da aula anterior se generalizam.
- ▶ Ponto fixo: condição $|g'| < 1$.
- ▶ Newton $\equiv$ ponto fixo com $g'(x^*) = 0$.

Aplicações Vistas

- ▶ ECC / Bitcoin.
- ▶ Fermi-Dirac / semicondutores.
- ▶ Van der Waals / engenharia química.
- ▶ Equação de Kepler.

Pontos de Atenção

- ▶ Escolha de g afeta convergência.
- ▶ Raízes múltiplas: usar método modificado.
- ▶ Sistemas: custo $O(n^3)$ por iteração.
- ▶ Otimização reduz-se a $f'(x) = 0$.

Ferramentas em Python (SciPy Developers, 2024)

```
scipy.optimize.fsolve  
scipy.optimize.brentq  
scipy.optimize.root  
scipy.optimize.minimize_scalar
```

- Kendall E. Atkinson. *An Introduction to Numerical Analysis*. Wiley, New York, 2 edition, 1989. Cap. 2: Rootfinding for Nonlinear Equations.
- Bitcoin Wiki. Secp256k1. Bitcoin Wiki, 2024. URL <https://en.bitcoin.it/wiki/Secp256k1>. Acesso em 2024.
- Richard L. Burden and J. Douglas Faires. *Numerical Analysis*. Brooks/Cole, Boston, MA, 9 edition, 2010. Cap. 2: Solutions of Equations in One Variable.
- Andrea Corbellini. Elliptic curve cryptography: A gentle introduction. Blog pessoal, 2015. URL <https://andrea.corbellini.name/2015/05/17/elliptic-curve-cryptography-a-gentle-introduction/>. Acesso em 2024.
- John M. A. Danby. *Fundamentals of Celestial Mechanics*. Willmann-Bell, Richmond, VA, 2 edition, 1988.
- Darrel Hankerson, Alfred Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer, New York, 2004.
- C. T. Kelley. *Solving Nonlinear Equations with Newton's Method*. Fundamentals of Algorithms. SIAM, Philadelphia, PA, 2003. URL <https://www.siam.org/books/fa01/>.

- Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987. doi: 10.2307/2007884.
- William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, Cambridge, 3 edition, 2007. URL <http://numerical.recipes/>. Cap. 9: Root Finding and Nonlinear Sets of Equations.
- SciPy Developers. SciPy optimization and root finding. Documentação oficial SciPy, 2024. URL <https://docs.scipy.org/doc/scipy/reference/optimize.html>. Acesso em 2024.
- Joseph M. Smith, Hendrick C. Van Ness, and Michael M. Abbott. *Introduction to Chemical Engineering Thermodynamics*. McGraw-Hill, New York, 7 edition, 2005.
- Simon M. Sze and Kwok K. Ng. *Physics of Semiconductor Devices*. Wiley, Hoboken, NJ, 3 edition, 2006.